

Gradient Decoupling for Inductive Temporal Link Prediction: A Controlled Study of Stateful Representations

Duong Viet Hoang^{a,*}, Duong Viet Huy^b, Lun-Min Shih^a

^a*Department of Computer Science and Information Engineering, Da-Yeh University*

No. 168, University Rd., Dacun, Changhua 515006, Taiwan

^b*Department of Computer Science, Kent State University*

P.O. Box 5190, Kent, OH 44242-0001, United States

**Corresponding author: Hoangduong4316@icloud.com*

ABSTRACT

Inductive temporal link prediction must accommodate entities absent from earlier observations. We study whether prediction gradients should update a stateful temporal representation. The model combines event encoding, recurrent node memory and deterministic pair histories with a shared state-transition readout. Coupled and decoupled training differ in whether the readout receives connected or detached features; a separate freeze-then-probe procedure tests a different optimization trajectory. Experiments on CoEdit, Wikipedia and MOOC use chronological partitions and matched random-destination sampling. Across three seeds, the decoupled model achieves mean inductive average precision of 0.9891, 0.9983 and 0.9947, respectively, compared with 0.9787, 0.9967 and 0.9910 for coupled training. Additional experiments examine predictor choice, compliance weighting, candidate-pool difficulty, backbone removal and frozen probes. Predictor replacement produces a larger performance change than the same-readout routing intervention, while learned representations outperform pair statistics alone. These findings support gradient routing as a design choice within the evaluated model. Their scope is limited by event-conditioned scoring, candidate pools that are not endpoint-type constrained, and mixed sequential and batch memory updates. CoEdit also shares its source with Wikipedia and includes retrospectively constructed attributes. We distinguish these restrictions from the measured routing effect and identify the evaluations needed for prospective forecasting.

Key words: temporal graphs, inductive link prediction, gradient decoupling, recurrent memory, reproducible evaluation

歸納式時序連結預測中的梯度解耦： 具狀態表徵的受控研究

Duong Viet Hoang^{a,*}, Duong Viet Huy^b, Lun-Min Shih^a

^a 大葉大學資訊工程學系

515006 臺灣彰化縣大村鄉學府路 168 號

^bDepartment of Computer Science, 肯特州立大學

P.O. Box 5190, Kent, OH 44242-0001, United States

* 通訊作者：Hoangduong4316@icloud.com

摘要

歸納式時序連結預測須處理先前觀測中未出現的節點。本研究探討預測梯度是否應更新具狀態的時序表徵。模型結合事件編碼、循環節點記憶及確定性節點對歷史，並使用共同的狀態轉移讀出器。耦合與解耦訓練的差異在於是否阻斷讀出器輸入的梯度；另以預訓練後凍結表徵、重新訓練預測頭的方法作為對照。研究採用 CoEdit、Wikipedia 與 MOOC 的時間切分及相同隨機目的端取樣。三個隨機種子下，解耦模型的平均歸納式平均精確率分別為 0.9891、0.9983 及 0.9947，耦合模型則為 0.9787、0.9967 及 0.9910。其他實驗分析預測器、相容性權重、候選池、骨幹移除及凍結探測。更換預測器造成的差異大於同一讀出器的梯度路由差異，而學習表徵的表現優於僅使用節點對統計量。結果支持在此模型中評估梯度路由的作用，但不代表端到端學習普遍較差。評估仍受事件條件式評分、未限制端點類型的候選池，以及混合循序與批次記憶更新所限。此外，CoEdit 與 Wikipedia 共享來源，部分屬性亦包含事後資訊。本文據此界定結果的適用範圍，並提出前瞻式預測所需的後續評估。

關鍵詞：時序圖，歸納式連結預測，梯度解耦，循環記憶，可重現評估

I. Introduction

Temporal link prediction uses ordered interactions to represent evolving relationships. A model must determine which observations enter its memory, how a candidate is represented, and which alternatives it must distinguish. These choices become especially important when test interactions involve entities absent from training and validation. Shared functions can transfer to such entities, but their usefulness depends on the retained history and candidate population.

Allowing prediction gradients to update a temporal encoder can improve task adaptation. It can also encourage features specific to the training population or sampled alternatives. Detaching the readout input changes this pressure without changing its forward value. The effect is empirical: removing a gradient may preserve useful information or suppress a necessary learning signal. Moreover, detached features are not static features. Recurrent memories continue to change as events arrive, and auxiliary objectives can still update backbone parameters.

We develop a stateful model combining event features, node memory and pair histories to examine this distinction. The principal comparison keeps the scoring architecture fixed and changes whether its input is detached. Additional experiments vary the predictor, compliance weights, candidate pools and learned backbone. A freeze-then-probe procedure evaluates whether a new prediction head can use a representation obtained through a different training trajectory. The contribution is a specified gradient-routing intervention and its empirical assessment, rather than a new claim for each constituent module. Source code and experimental summaries accompany the study [10].

II. Related Work

1. Temporal representations

GraphSAGE uses shared functions for inductive graph representation [16], while SEAL shows how candidate-centered sub-graphs can support link prediction [40]. Temporal graphs additionally require a representation of ordered history [18, 22]. DyRep models evolving representations and event occurrence [35]; JODIE learns user and item trajectories [26]; Temporal Graph Networks organize computation around messages and node memory [32]. Gated recurrent units provide one mechanism for updating such memories [7].

Other approaches use temporal neighborhood attention, as in TGAT [38], or ordered structural patterns, as in causal anony-

mous walks [36]. GraphMixer [8], NAT [28] and DyGFormer [39] offer alternative history encoders. Our model combines recurrent memory with pair statistics: an intensity-like recurrence inspired by Hawkes processes [17] and online moments computed by Welford’s method [37]. The gradient-routing result concerns this architecture and does not imply the same effect in these other models.

2. Representation adaptation

Probes assess information accessible from learned representations [1]. Fine-tuning can distort pretrained features under distribution shift [25], but that setting differs from a temporal model whose state evolves during evaluation. Stop-gradient also appears in BYOL [14] and SimSiam [6]; their self-supervised objectives do not determine its effect on supervised temporal links.

The information bottleneck [34] and variational formulations [2, 24] motivate representation regularization. However, deterministic representations require care in information-theoretic interpretation [3], and estimated information bounds have their own statistical limitations [30]. We therefore treat the KL term as an optimization component and do not infer removal of identity information from it.

3. Evaluation protocols

Negative sampling can change temporal model rankings, as demonstrated by the study introducing EdgeBank [31]. TGB [20], OGB [19] and TGB 2.0 [12] emphasize explicit datasets and evaluators; DyGLib facilitates shared dynamic-graph procedures [39]. More broadly, graph-learning and recommendation evaluations depend on comparator fidelity and tuning budgets [33, 11]. Model selection [5], seed variability [4] and input leakage [21] must therefore be considered alongside the metric. An inductive score alone neither identifies a shortcut [13] nor establishes transfer under deployment shift [9].

III. Problem and Model Specification

1. Events and recurrent representation

Let $e_i = (u_i, v_i, t_i, x_i)$ denote an observed interaction, with timestamp t_i and supplied attributes x_i . Events are ordered by nondecreasing time. Let $\mathcal{H}_i$ be the history retained by the current training or evaluation pass. The representation $h_i = F_\theta(e_i; \mathcal{H}_i)$ depends on this history and on the event attributes. A replacement destination provides a negative candidate conditioned on the same observed event.

The event encoder uses a residual transformation $\tilde{x} = x + \alpha_x f(x)$. The multilayer perceptron f maps the input feature dimension to 128 hidden units and back, with a ReLU activation. Its sigmoid gate combines source-node staleness with semantic novelty. Novelty is half the cosine distance from an exponential moving average of projected, normalized event features; the average uses decay 0.99 and is updated during training without gradients. The temporal gate takes $\log(1 + g)$, where g is source-node staleness.

Each node has a zero-initialized, 128-dimensional memory. Time-dependent gates decay the two endpoint memories. Two message networks combine these memories, a 128-dimensional pair context, a time encoding and $\tilde{x}$. Each message network has two 128-unit layers with a ReLU between them; a sigmoid function of the intensity-like signal scales its output. Separate GRU cells update source and destination memories using the opposite endpoint's message. The positive representation concatenates the two updated memories, giving $h_i \in \mathbb{R}^{256}$. The negative representation concatenates the updated source with the stored memory of the sampled destination. This asymmetry makes the task event-conditioned discrimination, rather than scoring all candidate pairs through identical hypothetical event updates.

A variational module maps stored node memories to diagonal Gaussian means and log variances, with log variance clipped to $[-4, 4]$. A learned gate blends a sampled latent vector with the original memory during training; evaluation uses the mean. A Gaussian KL penalty regularizes this module. Recurrent stores are detached between batches, so training does not backpropagate through the full event history.

2. Pair-history context

For each ordered pair, the model maintains recurrence, intensity, observation count, gap moments and a five-state distribution. The gap input is source-node staleness, not necessarily time since the pair's previous interaction. With k observations, the moments follow

$$\begin{aligned} \delta_k &= g_k - \mu_{k-1}, \quad \mu_k = \mu_{k-1} + \delta_k/k, \\ M_k &= M_{k-1} + \delta_k(g_k - \mu_k), \quad v_k = M_k/k. \end{aligned} \quad (1)$$

Recurrence and intensity evolve as

$$r_k = a + (1 - a)r_{k-1}e^{-\eta g_k}, \quad (2)$$

$$\ell_k = \mu_0 + (\ell_{k-1} - \mu_0)e^{-\beta g_k} + \alpha. \quad (3)$$

The constants are $a = 0.3$, $\eta = 0.001$, $\mu_0 = 0.1$, $\beta = 0.01$ and $\alpha = 1$. Fast and slow exponentially weighted averages of $1/\max(g_k, \epsilon)$ describe changes in activity. These quantities summarize history; ℓ_k is not interpreted as a calibrated event probability. A two-layer network combines event features and history signals to produce five transition logits. The resulting pair state is projected to the context used by the recurrent message networks.

The readout receives a seven-dimensional vector ϕ_i : intensity, log gap mean, log gap variance, recurrence, log source staleness, an accumulated probability of prior activity, and a standardized gap. These values use pre-update positive-pair history. The standardized gap is zero until two previous observations are available and is clipped to $[-5, 5]$. Negative candidates obtain their own pair histories through read-only lookup. Positive pair stores are updated within the forward pass before negative lookup, so a negative matching another positive pair in that batch can encounter a newer store state. The procedure does not provide globally identical pre-event snapshots for every candidate.

3. State-transition readout

The five states are idle, birth, reinforce, decay and death. A 256–128–5 observer with ReLU and softmax maps h to a distribution q . A low-rank transition operator combines a feature-dependent component with a pair-history bias:

$$T(h, \phi) = U(h)V(h) + A(\phi). \quad (4)$$

Here $U(h)$ and $V(h)$ have dimensions 5×3 and 3×5 . The network A maps 7 inputs through 14 ReLU units to 25 values. Its output layer starts at zero. A learnable mask $M = \sigma(P + \Delta)$ adjusts a fixed lifecycle prior P , with Δ initialized to zero. The next-state distribution is

$$\pi = \text{softmax}\left(q^T T + \log(q^T M + \epsilon) - b\right), \quad (5)$$

where b is zero except for the death coordinate, which equals five times the probability that the pair has never been active. The score is a clipped weighted sum,

$$p = \text{clip}\left(\sum_{c=1}^5 \text{softplus}(w_c)\pi_c, \epsilon, 1 - \epsilon\right). \quad (6)$$

The existence weights start at approximately $(0.1, 1, 1, 0.3, 0)$; $\epsilon = 10^{-6}$ bounds the score before conversion to a logit.

A separate hierarchical state calibration uses three 4–8–1 tanh gate networks for birth, continued activity and rising activity. Their outputs define a five-state tree distribution, which is constrained by the stored lifecycle state and used for auxiliary supervision and symbolic state updates. It is distinct from π in Eq. (6). Thus the hierarchy can affect subsequent history and training, but the reported AP is computed from the state-transition existence score.

4. Gradient routing and objective

Let $D_0(h) = h$ and $D_1(h) = \text{sg}(h)$, where stop-gradient preserves the value and has zero derivative. Both arms use $p = G_\psi(D_d(h), \phi)$ with the same readout. The connected arm has $d = 0$ and the detached arm $d = 1$. Through this input, the prediction-loss gradient is

$$\frac{\partial L_{\text{pred}}}{\partial \theta} = \frac{\partial L_{\text{pred}}}{\partial p} \frac{\partial p}{\partial D_d} \frac{\partial D_d}{\partial h} \frac{\partial h}{\partial \theta}. \quad (7)$$

Detachment removes this contribution for both positive and negative candidates. The pair-history inputs and compliance weights are also computed without gradients. Auxiliary losses still train their associated modules, and event-state updates continue in both arms. Routing also changes backbone gradients from auxiliary terms that reuse π ; the comparison therefore tests the shared input boundary rather than binary cross-entropy alone.

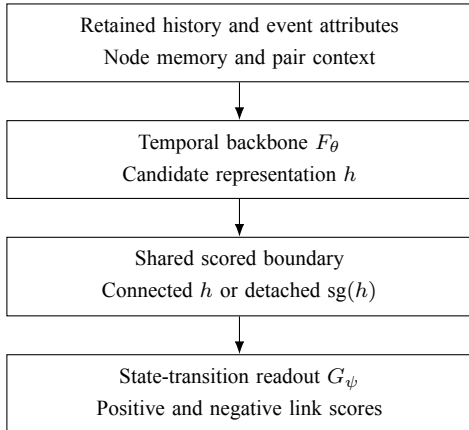


Figure 1. Scored feature path in the primary comparison.

Figure 1 shows the changed derivative boundary. Pair-history inputs also reach the readout directly; auxiliary objectives and store writes are omitted for clarity.

For a batch of B observed events and B sampled negatives,

prediction uses weighted binary cross-entropy:

$$L_{\text{pred}} = \frac{1}{2B} \sum_{j=1}^{2B} w_j \ell_j, \quad \ell_j = -y_j \log p_j - (1 - y_j) \log(1 - p_j). \quad (8)$$

Negative weights are one. Positive compliance is the product of three factors: a penalty for death without prior activity, $\exp(-3\|\pi - q\|_1)$ for abrupt state changes, and agreement between birth-plus-reinforce mass and normalized intensity. The product is clipped to $[0.05, 1]$, evaluated without gradients, and blended with unit weights after a two-epoch warmup over a three-epoch ramp.

The complete primary objective adds seven terms to L_{pred} . A Gaussian KL penalty, averaged over nodes and latent coordinates and weighted by the compression gate, has effective coefficient 10^{-5} . The mean death-without-prior-activity penalty has weight 0.05. KL divergence from a detached history-derived target to π has weight 0.1; the corresponding class-balanced KL to the calibrated state distribution has weight 0.5. Targets distinguish first occurrence, rising activity, declining activity and prolonged inactivity using counts, rate trends and standardized gaps. Inverse target-class frequencies are normalized to mean one within each batch. Finally, negative entropy of the batch-mean π has weight 0.01, while per-event negative entropy and KL to the uniform distribution of the continuous transition network have weights 0.02 and 0.01. The latter two terms differ only by an additive constant and together act as an entropy regularizer.

5. State evolution and frozen probes

Deterministic pair accumulators advance in stable event order, including repeated pairs within a batch. Neural memory candidates instead read a batch snapshot; when a node appears repeatedly, the final candidate in input order is committed. Batch size therefore affects the state approximation. This mixed procedure is shared by the primary arms and differs from fully sequential neural updates.

The freeze-then-probe procedure first trains a separate 256–128–1 ReLU prediction head jointly with the representation. After validation-based checkpoint selection, it freezes representation parameters and reinitializes that head. The new head is trained for a further 20 epochs, with its own validation selection. Temporal stores continue to evolve during this phase. This tests a different training trajectory, not an alternative implementation of the same detach intervention.

IV. Data and Experimental Protocol

1. Corpora and construction

Wikipedia and MOOC are bipartite interaction streams from the public JODIE sources [26]. Their endpoint roles have disjoint node-ID namespaces. CoEdit is a contributor-to-contributor graph derived from Wikipedia: distinct contributors editing the same page within 60 minutes form a pair, with averaged attributes and a midpoint timestamp. The builder retains the first 80,000 qualifying pairs in page first-observation order, then sorts them chronologically. This is not a global earliest-event sample.

CoEdit’s later edit is needed to construct an event timestamped at the midpoint. Its attributes are therefore not all prospectively available at that time. Moreover, CoEdit and Wikipedia share an underlying source and are not independent replications. Table 1 gives the actual corpus and split sizes; node counts include only observed identities.

Table 1. Corpus and chronological split statistics.

Quantity	CoEdit	Wikipedia	MOOC
Nodes	4,131	9,227	7,144
Events	80,000	157,474	411,749
Feature dimension	172	172	4
Training events	56,000	110,231	288,224
Validation events	12,000	23,621	61,762
Test events	12,000	23,622	61,763
Inductive nodes	535	916	227
Inductive test events	5,336	3,809	6,902

2. History and candidates

The chronological partition is 70/15/15. An inductive node occurs in testing but in neither training nor validation; an inductive event touches at least one such node. Training and validation history initialize the evaluation state. The inductive pass then processes only the filtered inductive test stream. Omitted test events do not update its memory. Repeated interactions with an inductive node can consequently supply history for later evaluated events.

Each positive receives one negative with the same source and a replacement destination. The pool follows the seen or unseen status of the true destination, and the true destination itself is rejected. However, candidates are not constrained to the destination role of a bipartite graph, and other real interactions at the same timestamp are not exhaustively excluded. Evaluation randomness is reset to support matched draws across primary arms. The resulting comparison is conditional on this sampled task;

near-ceiling AP does not establish performance on arbitrary future links.

3. Training and reporting

Table 2 lists the shared training settings. Optimization uses Adam [23] in PyTorch [29], with its weight-decay setting rather than AdamW’s decoupled update [27]. Validation selects checkpoints without using test AP. Runtime warmup restores parameters, optimizer state, random streams and temporal stores, so it does not add training epochs.

Table 2. Training settings for the primary comparison.

Setting	Value
Seeds	1, 7, 42
Training / probe epochs	20 / 20 after pretraining
Memory / pair dimensions	128 / 256
Batch size	500
Learning rate / weight decay	0.001 / 0.00001
Optimizer / schedule	Adam / cosine annealing
Readout	Shared state-transition head
Feature routing	Connected or detached
Compliance	Soft; floor 0.05
Evaluation negatives	One random destination per event

Average precision (AP) summarizes precision over recall increments. Tables report the arithmetic mean across seeds and the sample standard deviation with denominator $n - 1$. These deviations measure training-seed variability, not uncertainty across new graph populations. We make no significance claim from their overlap. Primary means use three seeds. The additional scoring-configuration study uses five seeds; its other experimental families use three. Additional experiments come from earlier configurations with incomplete original execution metadata, so their results are analyzed within each family and are not pooled with the primary comparison. Per-seed sources, checksums, pre-processing and aggregation scripts are provided with the repository.

V. Results

1. Same-readout routing comparison

Table 3 compares connected and detached features with the shared state-transition readout. The decoupled arm has higher mean inductive AP on each corpus. CoEdit shows the largest difference, whereas both arms are close to the upper bound on Wikipedia. These observations support a routing effect under the stated architecture and candidate protocol; they do not imply that

end-to-end learning is generally harmful.

Table 3. Inductive AP: mean and sample SD.

Corpus	Profile	AP
CoEdit	Coupled	0.9787 ± 0.0021
	Decoupled	0.9891 ± 0.0016
	Freeze then probe	0.8381 ± 0.0484
MOOC	Coupled	0.9910 ± 0.0047
	Decoupled	0.9947 ± 0.0007
	Freeze then probe	0.9862 ± 0.0009
Wikipedia	Coupled	0.9967 ± 0.0003
	Decoupled	0.9983 ± 0.0003
	Freeze then probe	0.9623 ± 0.0128

The frozen probe has lower mean AP than either primary arm. Its representation was pretrained with a different predictor and then exposed to a newly initialized head. Head capacity, training budget and the learned representation can all contribute to this outcome; it does not establish irreversible representation damage.

2. Predictor and compliance ablations

The CoEdit configuration study in Table 4 motivated the more specific routing comparison. The state head uses detached features, soft compliance and a 0.05 floor. The alternative combines a separate end-to-end MLP predictor with hard compliance gating and a zero floor. The TGAT-labeled comparator is a simplified proxy, so its result does not establish superiority over the published TGAT architecture.

Table 4. CoEdit scoring-configuration comparison.

Configuration	Inductive AP	Transductive AP
State head	0.9876 ± 0.0030	0.9985 ± 0.0003
MLP + hard gate	0.7593 ± 0.0140	0.9600 ± 0.0044
TGAT proxy	0.8466 ± 0.0133	0.8645 ± 0.0083

The difference between these scoring configurations is substantially larger than the same-readout difference in Table 3. It measures a broader architectural and optimization change. Table 5 separates predictor replacement, hard gating and a zero compliance floor. The final CoEdit variant combines all three. Wikipedia and MOOC compare the state head with the MLP predictor while retaining soft compliance.

Table 5. Predictor and gate ablations.

Corpus	Configuration	AP
CoEdit	State head	0.9899 ± 0.0016
	MLP predictor	0.7788 ± 0.0193
	Hard gate	0.9868 ± 0.0014
	Zero floor	0.9889 ± 0.0018
	Hard + zero	0.9870 ± 0.0021
	MLP + hard + zero	0.7798 ± 0.0221
Wikipedia	State head	0.9981 ± 0.0002
	MLP predictor	0.9490 ± 0.0072
MOOC	State head	0.9935 ± 0.0008
	MLP predictor	0.9844 ± 0.0017

Enabling the alternative predictor produces the largest reduction on CoEdit. Gate-only variants remain close to the state head, and combining all changes gives a result near predictor replacement alone. The state-head advantage also appears on Wikipedia and MOOC, with smaller differences. Predictor choice is therefore a major contributor to this contrast. Replacing it is a different intervention from reconnecting an otherwise unchanged state-transition readout.

3. Candidate-pool sensitivity

Table 6 evaluates each trained model under three candidate rules. The historical pool is a multiset of training destinations. The inductive pool is a multiset of destinations from test pairs absent from training. Each sampled destination is combined with the evaluated source. Thus historical does not require a previous interaction with that source, and inductive does not mean exclusively unseen nodes. These are repeated evaluations of a trained model under a fixed evaluation seed, not independent retrains.

Table 6. Candidate-pool sensitivity.

Corpus	Pool	State head	MLP
CoEdit	Random	0.9898 ± 0.0012	0.7899 ± 0.0545
	Historical	0.9573 ± 0.0031	0.5294 ± 0.0276
	Inductive	0.9625 ± 0.0024	0.5579 ± 0.0215
Wikipedia	Random	0.9968 ± 0.0005	0.9011 ± 0.0106
	Historical	0.9744 ± 0.0047	0.5585 ± 0.0147
	Inductive	0.9682 ± 0.0035	0.5423 ± 0.0032

Both configurations lose AP under harder pools, but the MLP loses more. The state head retains the higher mean in all reported corpus-pool combinations. This extends the predictor comparison beyond random destinations; it does not test the same-readout detach intervention under those harder pools. The inductive pool uses test membership for offline construction. Rejection of known evaluation positives has a bounded retry count, so resid-

ual false negatives remain possible and were not counted in the summaries.

4. Contribution of the learned backbone

Table 7 removes learned event encoding and recurrent-memory features while retaining a trained readout over deterministic pair statistics. The readout receives a zero vector in place of the learned pair representation. This comparison tests the learned backbone as a whole rather than each component separately.

Table 7. Learned-backbone contribution.

Corpus	Representation	Inductive AP
CoEdit	Full model	0.988326 ± 0.002338
	Pair statistics	0.920500 ± 0.001668
Wikipedia	Full model	0.996296 ± 0.000872
	Pair statistics	0.896506 ± 0.000004

The full representation has higher mean AP on both corpora. Pair statistics alone nevertheless retain substantial performance. These results support complementary roles for historical summaries and learned features, but do not isolate the event encoder, memory cells or auxiliary loss. The six-decimal precision preserves the small nonzero Wikipedia deviation in this experiment.

5. Additional frozen-probe experiments

Table 8 gives the earlier freeze-then-probe comparison. The Wikipedia probe with disjoint endpoint namespaces is labeled ID-corrected and kept separate from the earlier data version. It has no version-matched decoupled control in this experimental family.

Table 8. Freeze-then-probe comparison.

Corpus	Procedure	Inductive AP
CoEdit	Decoupled	0.9883 ± 0.0028
	Freeze then probe	0.7684 ± 0.0064
Wikipedia	Decoupled	0.9961 ± 0.0014
	Freeze then probe	0.9027 ± 0.0177
Wikipedia	Probe, ID-corrected	0.9513 ± 0.0030

The probe is below decoupled training in the earlier CoEdit and Wikipedia comparisons. The ID-corrected Wikipedia result differs from the earlier probe, illustrating the sensitivity to endpoint identity. Neither comparison proves that a pretrained representation cannot recover with a different head or budget. These retrospective results complement, but are not replications of, the primary probe in Table 3.

VI. Discussion

1. Meaning of the routing effect

The primary result identifies a difference associated with the gradient boundary in a fixed readout. Several explanations remain possible: detachment may preserve useful historical features, reduce adaptation to sampled candidates, or change the balance between prediction and auxiliary objectives. Aggregate AP does not distinguish them. In particular, identity-shortcut removal would require direct representation measurements and controlled interventions, not just favorable inductive performance.

The additional experiments clarify why broader configuration comparisons must be interpreted separately. Replacing a predictor changes its functional capacity and optimization; changing compliance modifies example weights; freezing after pretraining changes the representation’s trajectory. None estimates exactly the same intervention as detaching the shared readout input. Their combined value is to identify which design choices merit further controlled study.

2. Limits for forecasting and deployment

The evaluation scores observed-event representations against sampled alternatives. Positive and negative endpoint updates differ, candidate types are not restricted, and negative pair lookup can follow positive updates within the batch. Together with CoEdit’s retrospective attributes, these choices limit a prospective forecasting interpretation. A stronger forecasting evaluation would score every candidate from the same pre-event state, use only attributes available at the decision time, and restrict destinations to valid endpoint roles.

AP measures ranking under the sampled label balance. It does not establish probability calibration [15], nor justify deployment thresholds under a different event prevalence. Likewise, variation across three or five seeds cannot represent uncertainty across environments. CoEdit and Wikipedia share their source, and the simpler TGAT proxy does not provide a faithful external benchmark. The present evidence therefore supports an architectural comparison within this implementation, not a state-of-the-art claim.

3. Further evaluation

The next tests should apply the same-readout intervention to destination-valid, strictly checked candidate pools and fully pre-event scoring. Replication in faithful temporal architectures would assess whether the effect extends beyond this backbone.

Factorial ablations of event features, recurrent memory and pair statistics would separate their contributions. Probe studies should vary head capacity and recovery budget. These extensions would address the unresolved mechanisms without conflating gradient isolation, evolving event state and statistical generalization.

VII. Conclusion

We specify a stateful temporal-link model and compare connected and detached inputs to a shared state-transition readout. Decoupled training attains higher mean inductive AP on CoEdit, Wikipedia and MOOC under the evaluated protocol. Additional experiments show a larger sensitivity to predictor replacement, useful contributions from learned features, and lower performance for the tested frozen probes. The findings motivate testing gradient routing explicitly, while event-conditioned scoring, candidate validity and data construction constrain their use as evidence for prospective link forecasting.

References Cited

- [1] Alain, G. and Y. Bengio (2016) Understanding intermediate layers using linear classifier probes. arXiv:1610.01644.
- [2] Alemi, A. A., I. Fischer, J. V. Dillon and K. Murphy (2017) Deep variational information bottleneck. International Conference on Learning Representations. arXiv:1612.00410.
- [3] Amjad, R. A. and B. C. Geiger (2020) Learning Representations for Neural Network-Based Classification Using the Information Bottleneck Principle. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42, 2225-2239. doi:10.1109/tpami.2019.2909031.
- [4] Bouthillier, X., P. Delaunay, M. Bronzi, A. Trofimov, B. Nichyporuk, J. Szeto, et al. (2021) Accounting for Variance in Machine Learning Benchmarks. arXiv:2103.03098.
- [5] Cawley, G. C. and N. L. C. Talbot (2010) On Over-fitting in Model Selection and Subsequent Selection Bias in Performance Evaluation. *Journal of Machine Learning Research*, 11, 2079-2107.
- [6] Chen, X. and K. He (2021) Exploring simple Siamese representation learning. *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 15750-15758.
- [7] Cho, K., B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, et al. (2014) Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation. *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 1724-1734. doi:10.3115/v1/d14-1179.
- [8] Cong, W., S. Zhang, J. Kang, B. Yuan, H. Wu, X. Zhou, H. Tong and M. Mahdavi (2023) Do we really need complicated model architectures for temporal networks? *International Conference on Learning Representations*. arXiv:2302.11636.
- [9] D’Amour, A., K. Heller, D. Moldovan, B. Adlam, B. Alipanahi, A. Beutel, et al. (2020) Underspecification Presents Challenges for Credibility in Modern Machine Learning. arXiv:2011.03395.
- [10] Duong, V. H. and L.-M. Shih (2026) Temporal link decoupling: source, protocol and scientific evidence. Research repository. <https://github.com/hoangduong6210/temporal-link-decoupling>.
- [11] Ferrari Dacrema, M., P. Cremonesi and D. Jannach (2019) Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches. *Proceedings of the 13th ACM Conference on Recommender Systems*. doi:10.1145/3298689.3347058.
- [12] Gastinger, J., S. Huang, M. Galkin, E. Loghmani, A. Parviz, F. Poursafaei, et al. (2024) TGB 2.0: A Benchmark for Learning on Temporal Knowledge Graphs and Heterogeneous Graphs. arXiv:2406.09639.
- [13] Geirhos, R., J. Jacobsen, C. Michaelis, R. Zemel, W. Brendel, M. Bethge, et al. (2020) Shortcut Learning in Deep Neural Networks. arXiv:2004.07780.
- [14] Grill, J., F. Strub, F. Altché, C. Tallec, P. H. Richemond, E. Buchatskaya, et al. (2020) Bootstrap your own latent: A new approach to self-supervised Learning. arXiv:2006.07733.
- [15] Guo, C., G. Pleiss, Y. Sun and K. Q. Weinberger (2017) On Calibration of Modern Neural Networks. arXiv:1706.04599.
- [16] Hamilton, W. L., R. Ying and J. Leskovec (2017) Inductive Representation Learning on Large Graphs. arXiv:1706.02216.
- [17] Hawkes, A. G. (1971) Spectra of some self-exciting and mutually exciting point processes. *Biometrika*, 58(1), 83-90.

- [18] Holme, P. and J. Saramäki (2012) Temporal Networks. *Physics Reports*, 519(3), 97–125. doi:10.1016/j.physrep.2012.03.001.
- [19] Hu, W., M. Fey, M. Zitnik, Y. Dong, H. Ren, B. Liu, et al. (2020) Open Graph Benchmark: Datasets for Machine Learning on Graphs. arXiv:2005.00687.
- [20] Huang, S., F. Poursafaei, J. Danovitch, M. Fey, W. Hu, E. Rossi, J. Leskovec, M. Bronstein, G. Rabusseau and R. Rabbany (2023) Temporal graph benchmark for machine learning on temporal graphs. *Advances in Neural Information Processing Systems*, 36. arXiv:2307.01026.
- [21] Kapoor, S. and A. Narayanan (2022) Leakage and the Reproducibility Crisis in ML-based Science. arXiv:2207.07048.
- [22] Kazemi, S. M., R. Goel, K. Jain, I. Kobyzev, A. Sethi, P. Forsyth, et al. (2019) Representation Learning for Dynamic Graphs: A Survey. arXiv:1905.11485.
- [23] Kingma, D. P. and J. Ba (2015) Adam: A method for stochastic optimization. *International Conference on Learning Representations*. arXiv:1412.6980.
- [24] Kingma, D. P. and M. Welling (2014) Auto-encoding variational Bayes. *International Conference on Learning Representations*. arXiv:1312.6114.
- [25] Kumar, A., A. Raghunathan, R. Jones, T. Ma and P. Liang (2022) Fine-tuning can distort pretrained features and underperform out-of-distribution. *International Conference on Learning Representations*. arXiv:2202.10054.
- [26] Kumar, S., X. Zhang and J. Leskovec (2019) Predicting dynamic embedding trajectory in temporal interaction networks. *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 1269–1278.
- [27] Loshchilov, I. and F. Hutter (2017) Decoupled Weight Decay Regularization. arXiv:1711.05101.
- [28] Luo, Y. and P. Li (2022) Neighborhood-aware scalable temporal network representation learning. *Proceedings of the First Learning on Graphs Conference*, PMLR, 198, 1:1–1:18.
- [29] Paszke, A., S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, et al. (2019) PyTorch: An Imperative Style, High-Performance Deep Learning Library. arXiv:1912.01703.
- [30] Poole, B., S. Ozair, A. van den Oord, A. A. Alemi and G. Tucker (2019) On Variational Bounds of Mutual Information. arXiv:1905.06922.
- [31] Poursafaei, F., S. Huang, K. Pelrine and R. Rabbany (2022) Towards better evaluation for dynamic link prediction. *Advances in Neural Information Processing Systems*, 35. arXiv:2207.10128.
- [32] Rossi, E., B. Chamberlain, F. Frasca, D. Eynard, F. Monti and M. Bronstein (2020) Temporal graph networks for deep learning on dynamic graphs. *ICML Workshop on Graph Representation Learning and Beyond*. arXiv:2006.10637.
- [33] Shchur, O., M. Mumme, A. Bojchevski and S. Günnemann (2018) Pitfalls of Graph Neural Network Evaluation. arXiv:1811.05868.
- [34] Tishby, N., F. C. Pereira and W. Bialek (1999) The information bottleneck method. *Proceedings of the Allerton Conference on Communication, Control and Computing*. Preprint: arXiv:physics/0004057.
- [35] Trivedi, R., M. Farajtabar, P. Biswal and H. Zha (2019) DyRep: Learning representations over dynamic graphs. *International Conference on Learning Representations*.
- [36] Wang, Y., Y.-Y. Chang, Y. Liu, J. Leskovec and P. Li (2021) Inductive representation learning in temporal networks via causal anonymous walks. *International Conference on Learning Representations*. arXiv:2101.05974.
- [37] Welford, B. P. (1962) Note on a method for calculating corrected sums of squares and products. *Technometrics*, 4(3), 419–420.
- [38] Xu, D., C. Ruan, E. Korpoglu, S. Kumar and K. Achan (2020) Inductive representation learning on temporal graphs. *International Conference on Learning Representations*. arXiv:2002.07962.
- [39] Yu, L., L. Sun, B. Du and W. Lv (2023) Towards better dynamic graph learning: New architecture and unified library. *Advances in Neural Information Processing Systems*, 36. arXiv:2303.13047.
- [40] Zhang, M. and Y. Chen (2018) Link Prediction Based on Graph Neural Networks. arXiv:1802.09691.